

Classical Machine Learning for Computer Vision

K-Means, K-Nearest Neighbours, SVM, Decision Trees, and Bag-of-Visual-Words

Contents

1	Introduction: From Images to Classical Machine Learning	4
2	Feature Space: What the Classifier Actually Sees	4
3	K-Means Clustering	5
3.1	Purpose and CV Motivation	5
3.2	Mathematical Objective	5
3.3	Assignment Step	6
3.4	Centroid Update and Its Derivation	6
3.5	K-Means Algorithm: Pseudocode	7
3.6	K-Means++ Initialization	7
3.7	What K-Means Means Geometrically	7
3.8	CV Example: Building a Visual Dictionary	7
3.9	Advantages and Limitations	8
4	K-Nearest Neighbours (KNN)	8
4.1	Core Idea	8
4.2	Why K Matters	8
4.3	Distance-Weighted KNN	8
4.4	KNN Pseudocode	9
4.5	Distance Functions and Their Mathematical Meaning	9
4.5.1	Euclidean Distance (L_2)	9
4.5.2	Squared Euclidean Distance	9
4.5.3	Manhattan Distance (L_1)	9
4.5.4	Minkowski Distance	10
4.5.5	Chebyshev Distance (L_∞)	10
4.5.6	Cosine Similarity / Cosine Distance	10
4.5.7	Hamming Distance	10
4.6	Feature Scaling: A Critical KNN Issue	11
4.7	KNN in CV: When to Use It	11
5	Support Vector Machine (SVM)	11
5.1	Motivation	11
5.2	Geometric Meaning of the Weight Vector and Bias	12
5.3	Margin	12
5.4	Hard-Margin SVM	12
5.5	Soft-Margin SVM and Slack Variables	12
5.6	Hinge Loss Interpretation	13
5.7	Hinge Loss in Detail: How and Why?	13
5.8	Why Hinge Loss Produces a Maximum-Margin Classifier	13

5.9	Stochastic Gradient Descent (SGD)	14
5.10	SGDClassifier with Hinge Loss in scikit-learn	15
5.11	SGD Training Algorithm for a Linear Hinge-Loss Classifier	15
5.12	SVM vs SGDClassifier(loss="hinge") vs LinearSVC	16
5.13	Support Vectors	16
5.14	SVM Training: Conceptual Pseudocode	16
5.15	SVM Inference	17
6	Kernel SVM: Why and When?	17
6.1	Why a Kernel Is Needed	17
6.2	Kernel Trick	17
6.3	Dual Form and Kernelized Prediction	17
6.4	Common Kernels	18
6.5	When Should a Kernel Be Used?	18
7	Multiclass and Multilabel SVM	18
7.1	Multiclass Classification	18
7.2	Multilabel Classification	19
8	Decision Trees	19
8.1	Motivation in Computer Vision	19
8.2	Continuous vs Discrete Features	19
8.3	Node Impurity: Gini Index	20
8.4	Entropy	20
8.5	Information Gain	20
8.6	Tree Training Algorithm	20
8.7	Geometric Interpretation	21
8.8	Advantages, Limitations, and CV Use	21
9	Bag-of-Visual-Words (BoVW)	22
9.1	Why BoVW?	22
9.2	Complete Pipeline	22
9.3	Step 1: Extract Local Descriptors	22
9.4	Step 2: Learn the Visual Dictionary	22
9.5	Step 3: Vector Quantization	22
9.6	Step 4: Histogram Construction	23
9.7	Histogram Normalization	23
9.8	Step 5: Classification	23
9.9	BoVW Pseudocode	23
9.10	Major Limitation: Loss of Spatial Information	24
10	Comparison of Classical Classifiers for Computer Vision	24
11	Which Method Should Be Used When?	24
11.1	KNN	24
11.2	SVM	24
11.3	Decision Tree	25
11.4	K-Means	25
12	Feature-Space Requirements: A Practical View	25
13	Training Cost vs Inference Cost	25

14 End-to-End Classical CV Example	25
15 Important Practical Lessons for Computer Vision	26
16 Summary Table	27
17 Final Conceptual Picture	27

1 Introduction: From Images to Classical Machine Learning

A classical computer vision recognition system separates two questions:

1. **How should the image be represented?**
2. **How should classes be separated using that representation?**

A typical pipeline is

Image $\rightarrow$ Feature detection $\rightarrow$ Feature description
 $\rightarrow$ Feature vector $\rightarrow$ Machine learning $\rightarrow$ Prediction

For example,

Image $\rightarrow$ SIFT $\rightarrow$ local descriptors $\rightarrow$ Bag of Visual Words $\rightarrow$ SVM $\rightarrow$ class.

Key idea: feature extraction determines the representation;
ML determines how classes are separated in that representation.

The mathematical notation used throughout these notes is summarized below.

Notation	Meaning	Significance in CV
N	Number of samples	Number of images, patches, or descriptors
d	Feature dimension	Number of numerical values used to describe one sample
C	Number of classes	Number of recognition categories
K	Context-dependent integer	Number of clusters in K-Means or neighbours in KNN
$\mathbf{x}_i \in \mathbb{R}^d$	i -th feature vector	One image/descriptor represented as a point in d -dimensional space
y_i	Label of sample i	Ground-truth class used for supervised learning
$\mathcal{D}$	Dataset	Collection of training pairs $(\mathbf{x}_i, y_i)$
$\ \cdot\ _2$	Euclidean norm	Measures geometric distance between continuous feature vectors
$\arg \min / \arg \max$	Argument giving minimum/maximum	Selects the best cluster, neighbour class, or model output
$\sum$	Summation	Aggregates errors, distances, votes, or losses
$\mathbb{I}[P]$	Indicator function	Equals 1 when statement P is true and 0 otherwise

2 Feature Space: What the Classifier Actually Sees

Suppose an image is converted into d numerical features. We write

$$\mathbf{x}_i = [x_{i1}, x_{i2}, \dots, x_{id}]^T \in \mathbb{R}^d.$$

Here:

- i identifies the sample/image;
- j identifies a feature coordinate;
- x_{ij} is the value of feature j for image i ;
- d is the total number of feature coordinates.

Thus an image is no longer treated as pixels by the classical classifier; it is treated as a point in a feature space.

For supervised classification,

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N, \quad y_i \in \{1, 2, \dots, C\}.$$

The classifier attempts to learn

$$f : \mathbb{R}^d \rightarrow \{1, \dots, C\}.$$

The nature of f is what differentiates KNN, SVM, and decision trees.

Method	What it assumes	What it learns
K-Means	Similar points should form compact groups	Cluster centroids
KNN	Nearby samples should have similar labels	Essentially the stored training examples
SVM	A good classifier should separate classes with a large margin	A decision boundary
Decision Tree	Useful thresholds can recursively partition the feature space	A hierarchy of feature tests

3 K-Means Clustering

3.1 Purpose and CV Motivation

K-Means is an **unsupervised** learning algorithm: no class labels are required. Its goal is to divide N vectors into K groups.

In computer vision, an important use is learning a **visual dictionary**. If thousands or millions of local SIFT descriptors are extracted from training images, K-Means groups similar descriptors. Each centroid becomes a *visual word*.

Let the descriptor dimension be $d = 128$ for standard SIFT. A descriptor is therefore

$$\mathbf{x}_i \in \mathbb{R}^{128}.$$

The dictionary contains K visual words

$$\mathcal{V} = \{\mathbf{v}_1, \dots, \mathbf{v}_K\}, \quad \mathbf{v}_k \in \mathbb{R}^{128}.$$

3.2 Mathematical Objective

Each point receives a cluster index

$$z_i \in \{1, 2, \dots, K\}.$$

The centroid of cluster k is $\boldsymbol{\mu}_k \in \mathbb{R}^d$. K-Means minimizes

$$J = \sum_{i=1}^N \|\mathbf{x}_i - \boldsymbol{\mu}_{z_i}\|_2^2.$$

Meaning of every symbol.

- J : total within-cluster squared error; smaller means points lie closer to their assigned centroids.
- $\mathbf{x}_i$: feature vector being clustered.

- μ_{z_i} : centroid of the cluster currently assigned to x_i .
- $\|x_i - \mu_{z_i}\|_2$: Euclidean distance between the point and centroid.
- Square: makes large deviations more costly and gives the standard least-squares K-Means objective.

Equivalently,

$$J = \sum_{k=1}^K \sum_{i:z_i=k} \|x_i - \mu_k\|_2^2.$$

The inner sum measures the error inside cluster k ; the outer sum adds errors from all clusters.

3.3 Assignment Step

For fixed centroids, assign each point to its nearest centroid:

$$z_i = \arg \min_{k \in \{1, \dots, K\}} \|x_i - \mu_k\|_2^2.$$

The notation $\arg \min_k$ means “return the value of k that gives the smallest expression.”

3.4 Centroid Update and Its Derivation

Let C_k denote the set of points assigned to cluster k . We want the centroid that minimizes

$$J_k = \sum_{i \in C_k} \|x_i - \mu_k\|_2^2.$$

Taking the derivative with respect to the vector μ_k ,

$$\frac{\partial J_k}{\partial \mu_k} = -2 \sum_{i \in C_k} (x_i - \mu_k).$$

Setting the derivative to zero,

$$\sum_{i \in C_k} x_i - |C_k| \mu_k = 0,$$

so

$$\mu_k = \frac{1}{|C_k|} \sum_{i \in C_k} x_i.$$

Here $|C_k|$ means the number of samples in cluster k . Thus the centroid is literally the arithmetic mean of all points assigned to that cluster.

3.5 K-Means Algorithm: Pseudocode

Algorithm 1 K-Means Clustering

Require: Dataset $\mathcal{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$, number of clusters K

Ensure: Centroids $\{\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_K\}$ and assignments $\{z_1, \dots, z_N\}$

```
1: Initialize  $\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_K$ 
2: repeat
3:   for  $i = 1, \dots, N$  do
4:      $z_i \leftarrow \arg \min_{k \in \{1, \dots, K\}} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|_2^2$ 
5:   end for
6:   for  $k = 1, \dots, K$  do
7:      $\boldsymbol{\mu}_k \leftarrow \frac{\sum_{i=1}^N \mathbf{1}[z_i = k] \mathbf{x}_i}{\sum_{i=1}^N \mathbf{1}[z_i = k]}$ 
8:   end for
9: until  $J = \sum_{i=1}^N \|\mathbf{x}_i - \boldsymbol{\mu}_{z_i}\|_2^2$  converges
10: return  $\{\boldsymbol{\mu}_k\}, \{z_i\}, J$ 
```

Typical stopping conditions are: assignments stop changing, centroid movement becomes smaller than a tolerance, or a maximum number of iterations is reached.

3.6 K-Means++ Initialization

Poor random initialization can lead to a poor local solution. K-Means++ spreads initial centroids across the data.

After selecting some centroids, define

$$D(\mathbf{x}_i) = \min_k \|\mathbf{x}_i - \boldsymbol{\mu}_k\|_2.$$

Then choose a new centre with probability

$$P(\mathbf{x}_i) = \frac{D(\mathbf{x}_i)^2}{\sum_j D(\mathbf{x}_j)^2}.$$

Points far from existing centres have larger probability of becoming initial centres.

3.7 What K-Means Means Geometrically

K-Means creates a Voronoi-like partition of feature space: every point is assigned to its nearest centroid. Therefore, it works best when clusters are reasonably compact and approximately spherical under the selected distance measure.

3.8 CV Example: Building a Visual Dictionary

Suppose 10,000 training images produce 2 million SIFT descriptors. Each descriptor is 128-dimensional. Running K-Means with $K = 500$ produces

$$500 \text{ centroids} \in \mathbb{R}^{128}.$$

These 500 centroids form the visual vocabulary. A new SIFT descriptor is assigned to its nearest centroid, turning a continuous descriptor into a discrete visual-word index from 1 to 500.

3.9 Advantages and Limitations

- **Advantages:** simple; scalable variants exist; intuitive; useful for vector quantization and BoVW.
- **Limitations:** K must be selected; sensitive to initialization; affected by feature scaling and outliers; Euclidean geometry may not match the true descriptor similarity; optimization is non-convex.

4 K-Nearest Neighbours (KNN)

4.1 Core Idea

KNN is a supervised, non-parametric, instance-based classifier. It assumes that points close to each other in feature space tend to have similar labels.

Given a query vector $\mathbf{x}$, compute its distance to every training vector $\mathbf{x}_i$:

$$d_i = D(\mathbf{x}, \mathbf{x}_i).$$

Sort the distances and retain the K smallest. Let $\mathcal{N}_K(\mathbf{x})$ denote the set of those K neighbours.

The majority-vote classifier is

$$\hat{y} = \arg \max_c \sum_{i \in \mathcal{N}_K(\mathbf{x})} \mathbf{1}[y_i = c].$$

Here $\hat{y}$ is the predicted class, c is a candidate class, and the indicator contributes 1 for a neighbour belonging to c and 0 otherwise.

4.2 Why K Matters

- Small K : very local decision; low bias but high sensitivity to noise.
- Large K : smoother decision; more robust to noise but may blur class boundaries.

Thus K controls the bias–variance trade-off.

4.3 Distance-Weighted KNN

A common idea is to give closer neighbours more influence:

$$w_i = \frac{1}{d_i + \epsilon},$$

where $\epsilon > 0$ prevents division by zero. Then

$$\hat{y} = \arg \max_c \sum_{i \in \mathcal{N}_K(\mathbf{x})} w_i \mathbf{1}[y_i = c].$$

4.4 KNN Pseudocode

Algorithm 2 K -Nearest Neighbours Classification

Require: $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, query $\mathbf{x}$, K , distance D

Ensure: Predicted class $\hat{y}$

```
1: for  $i = 1, \dots, N$  do
2:    $d_i \leftarrow D(\mathbf{x}, \mathbf{x}_i)$ 
3: end for
4:  $\mathcal{N}_K(\mathbf{x}) \leftarrow$  indices of the  $K$  smallest distances
5: for each class  $c$  do
6:    $V_c \leftarrow \sum_{i \in \mathcal{N}_K(\mathbf{x})} \mathbf{1}[y_i = c]$ 
7: end for
8:  $\hat{y} \leftarrow \arg \max_c V_c$ 
9: return  $\hat{y}$ 
```

4.5 Distance Functions and Their Mathematical Meaning

A distance function defines what “similar” means. This is crucial in KNN because changing the distance changes the neighbourhood and therefore can change the classification result.

4.5.1 Euclidean Distance (L_2)

$$D_2(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{j=1}^d (x_j - y_j)^2}.$$

x_j and y_j are the values of feature coordinate j . The difference is squared, summed, and square-rooted.

Significance: geometrically this is the ordinary straight-line distance in feature space. It is a strong default for dense continuous features such as many normalized handcrafted descriptors.

Pros: intuitive; rotationally invariant; well understood.

Cons: sensitive to feature scale; in high dimensions distances can become less discriminative.

4.5.2 Squared Euclidean Distance

$$D_2^2(\mathbf{x}, \mathbf{y}) = \sum_{j=1}^d (x_j - y_j)^2.$$

Because square root is monotonic,

$$D_2(\mathbf{x}, \mathbf{y}) < D_2(\mathbf{x}, \mathbf{y}') \iff D_2^2(\mathbf{x}, \mathbf{y}) < D_2^2(\mathbf{x}, \mathbf{y}').$$

Therefore it gives the same nearest-neighbour ordering while avoiding the square-root operation.

4.5.3 Manhattan Distance (L_1)

$$D_1(\mathbf{x}, \mathbf{y}) = \sum_{j=1}^d |x_j - y_j|.$$

Instead of squaring deviations, it adds absolute deviations.

Pros: less dominated by a few large coordinate errors; can work well for sparse or heavy-tailed representations.

Cons: still sensitive to scale; not automatically superior to L_2 .

4.5.4 Minkowski Distance

$$D_p(\mathbf{x}, \mathbf{y}) = \left(\sum_{j=1}^d |x_j - y_j|^p \right)^{1/p}.$$

p controls how coordinate differences are aggregated. $p = 1$ gives Manhattan distance and $p = 2$ gives Euclidean distance.

Pros: flexible family.

Cons: p becomes another hyperparameter and high-dimensional behaviour still needs consideration.

4.5.5 Chebyshev Distance (L_∞)

$$D_\infty(\mathbf{x}, \mathbf{y}) = \max_j |x_j - y_j|.$$

Only the largest coordinate deviation determines the distance.

Pros: useful when the worst individual coordinate error matters.

Cons: rarely the best choice for standard CV feature descriptors.

4.5.6 Cosine Similarity / Cosine Distance

$$\text{sim}_{\cos}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x}^T \mathbf{y}}{\|\mathbf{x}\|_2 \|\mathbf{y}\|_2}, \quad D_{\cos} = 1 - \text{sim}_{\cos}.$$

The numerator is the dot product. The denominator normalizes by vector lengths. Thus cosine similarity measures the angle between vectors rather than their absolute magnitude.

Pros: useful for normalized histograms and directional representations; insensitive to overall scale.

Cons: magnitude information is discarded; zero vectors require special handling.

4.5.7 Hamming Distance

For binary vectors $\mathbf{x}, \mathbf{y} \in \{0, 1\}^d$,

$$D_H(\mathbf{x}, \mathbf{y}) = \sum_{j=1}^d \mathbf{1}[x_j \neq y_j].$$

Equivalently, bitwise XOR followed by population count can compute it efficiently.

CV significance: natural for binary descriptors such as BRIEF, ORB, and BRISK.

Pros: extremely fast; hardware-friendly.

Cons: inappropriate for ordinary real-valued SIFT descriptors.

Distance	Typical representation	Advantages	Limitations
L_2	Dense continuous vectors	Intuitive; strong baseline	Scale and high-dimensional sensitivity
L_1	Continuous/sparse vectors	Less affected by large coordinate errors	Scale sensitive
Minkowski	General numerical features	Flexible through p	Requires choosing p
Cosine	Normalized vectors/histograms	Focuses on direction	Discards magnitude
Hamming	Binary descriptors	Very fast	Binary data only
Chebyshev	Worst-coordinate criterion	Simple; interpretable	Uncommon in standard CV

Table 1: Practical comparison of common KNN distances.

4.6 Feature Scaling: A Critical KNN Issue

Suppose one feature ranges from 0 to 1 while another ranges from 0 to 10,000. Under Euclidean distance, the second feature can dominate:

$$D_2^2 = (x_1 - y_1)^2 + (x_2 - y_2)^2.$$

A common remedy is standardization:

$$z_j = \frac{x_j - \mu_j}{\sigma_j},$$

where μ_j and σ_j are the training-set mean and standard deviation of feature j .

4.7 KNN in CV: When to Use It

KNN is attractive for small-to-moderate datasets, simple baselines, and cases where a meaningful feature-space distance is available. Its major weakness is inference cost: a query may need distances to many training samples.

5 Support Vector Machine (SVM)

5.1 Motivation

SVM is a supervised classifier designed to find a decision boundary with a large margin. For two classes, encode labels as

$$y_i \in \{-1, +1\}.$$

A linear classifier is

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b,$$

where $\mathbf{w} \in \mathbb{R}^d$ is the weight vector and $b \in \mathbb{R}$ is the bias/intercept.

The predicted class is

$$\hat{y} = \text{sign}(\mathbf{w}^T \mathbf{x} + b).$$

5.2 Geometric Meaning of the Weight Vector and Bias

The decision boundary is

$$\mathbf{w}^T \mathbf{x} + b = 0.$$

The vector $\mathbf{w}$ is perpendicular (normal) to this hyperplane. Therefore it controls the orientation of the boundary. The scalar b shifts the boundary.

The signed distance from a point $\mathbf{x}$ to the hyperplane is

$$\frac{\mathbf{w}^T \mathbf{x} + b}{\|\mathbf{w}\|_2}.$$

Thus the sign tells which side of the boundary the point lies on, while the magnitude gives distance from the boundary.

5.3 Margin

For a canonical SVM solution, the two supporting hyperplanes are

$$\mathbf{w}^T \mathbf{x} + b = +1, \quad \mathbf{w}^T \mathbf{x} + b = -1.$$

Their distance is

$$\text{Margin} = \frac{2}{\|\mathbf{w}\|_2}.$$

Hence maximizing margin is equivalent to minimizing $\|\mathbf{w}\|_2^2$.

5.4 Hard-Margin SVM

For perfectly separable data,

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1, \quad \forall i.$$

The optimization problem is

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|_2^2 \quad \text{subject to} \quad y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1.$$

Meaning. The constraint forces every training point onto the correct side of the margin. The objective prefers the smallest $\|\mathbf{w}\|$, which gives the largest margin.

5.5 Soft-Margin SVM and Slack Variables

Real CV data are rarely perfectly separable. Introduce slack variables $\xi_i \geq 0$:

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i.$$

The optimization becomes

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_{i=1}^N \xi_i.$$

Here $C > 0$ controls the trade-off between a large margin and training violations.

Interpretation:

- large C : penalize violations strongly; training errors are less tolerated;
- small C : allow more violations in exchange for a wider margin.

5.6 Hinge Loss Interpretation

The same idea can be written using hinge loss:

$$\ell_i = \max(0, 1 - y_i(\mathbf{w}^T \mathbf{x}_i + b)).$$

The quantity $y_i(\mathbf{w}^T \mathbf{x}_i + b)$ is called the **margin score** for sample i . If it is at least 1, hinge loss is zero. If it is below 1, a penalty is incurred.

5.7 Hinge Loss in Detail: How and Why?

The hinge loss is central to understanding why a linear SVM can also be trained with stochastic gradient descent. For a training sample $(\mathbf{x}_i, y_i)$, define the **signed margin score**

$$m_i = y_i(\mathbf{w}^T \mathbf{x}_i + b).$$

Here $y_i \in \{-1, +1\}$ flips the sign of the classifier score so that a correctly classified point has $m_i > 0$. The magnitude tells us how confidently the point lies on the correct side.

The hinge loss is

$$\ell_{\text{hinge}}(m_i) = \max(0, 1 - m_i).$$

The number 1 is the desired canonical margin. The loss therefore behaves as follows:

Margin score m_i	Hinge loss	Interpretation
$m_i \geq 1$	0	Correct and outside the margin
$0 < m_i < 1$	$1 - m_i$	Correct, but inside the margin
$m_i = 0$	1	On the decision boundary
$m_i < 0$	$1 - m_i > 1$	Misclassified

This is important: the hinge loss does not merely ask whether the point is correctly classified. It also penalizes points that are correctly classified but too close to the decision boundary. This encourages a safety margin.

Example. Suppose $y_i = +1$ and the classifier produces $\mathbf{w}^T \mathbf{x}_i + b = 2$. Then

$$m_i = (+1)(2) = 2, \quad \ell_i = \max(0, 1 - 2) = 0.$$

The point is correctly classified and lies beyond the required margin. If instead the score is 0.4,

$$m_i = 0.4, \quad \ell_i = 1 - 0.4 = 0.6.$$

The point is correctly classified, but it is inside the margin and therefore contributes to training. If the score is -0.5 ,

$$m_i = -0.5, \quad \ell_i = 1 - (-0.5) = 1.5,$$

so the classifier receives an even stronger penalty.

5.8 Why Hinge Loss Produces a Maximum-Margin Classifier

The soft-margin SVM objective can be expressed as a regularized empirical-risk problem:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_{i=1}^N \max(0, 1 - y_i(\mathbf{w}^T \mathbf{x}_i + b)).$$

The two terms have different jobs:

- $\frac{1}{2}\|\mathbf{w}\|_2^2$ controls model complexity and, geometrically, encourages a large margin;
- the hinge-loss sum measures violations of the desired margin;
- C controls how strongly margin violations are penalized relative to regularization.

Thus the SVM is balancing two goals: **keep the boundary simple/wide** and **fit the training examples well**.

The hinge loss is also convex and piecewise linear. Its subgradient with respect to $\mathbf{w}$ for one sample is

$$\frac{\partial \ell_i}{\partial \mathbf{w}} = \begin{cases} -y_i \mathbf{x}_i, & y_i(\mathbf{w}^T \mathbf{x}_i + b) < 1, \\ 0, & y_i(\mathbf{w}^T \mathbf{x}_i + b) > 1. \end{cases}$$

At exactly $m_i = 1$, the function has a corner; a subgradient can be used. The practical consequence is simple: **points outside the margin stop contributing hinge-loss gradient, while points inside or on the wrong side continue to push the boundary**.

5.9 Stochastic Gradient Descent (SGD)

Gradient descent minimizes an objective by repeatedly moving the parameters opposite to the gradient. If the objective is $J(\theta)$, where θ represents the learnable parameters, the basic update is

$$\theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta).$$

Here $\eta > 0$ is the learning rate and $\nabla_{\theta} J$ is the direction of steepest increase of the objective.

For a dataset of N samples, ordinary (batch) gradient descent computes the gradient using all N samples before making one update. SGD instead uses one randomly selected sample at a time (or a small mini-batch), giving a cheap noisy estimate of the full gradient.

For the hinge-loss SVM, ignoring the nonessential constant factors, one sample contributes

$$J_i(\mathbf{w}, b) = \frac{\lambda}{2} \|\mathbf{w}\|_2^2 + \max(0, 1 - y_i(\mathbf{w}^T \mathbf{x}_i + b)),$$

where λ controls regularization in this parameterization. If the sample violates the margin,

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) < 1,$$

the update contains the hinge-loss contribution. A typical subgradient update is

$$\mathbf{w} \leftarrow \mathbf{w} - \eta [\lambda \mathbf{w} - y_i \mathbf{x}_i]$$

and

$$b \leftarrow b + \eta y_i$$

for a margin-violating sample. For a sample satisfying $m_i > 1$, only the regularization part contributes.

Physical intuition. Imagine moving a separating line through a cloud of CV feature vectors. A point that is already far on the correct side does not need to influence the boundary. A point close to the boundary, or on the wrong side, produces a non-zero hinge-loss gradient and pushes the parameters. Repeating these small corrections over many samples gradually moves the hyperplane toward a useful maximum-margin solution.

5.10 SGDClassifier with Hinge Loss in scikit-learn

In scikit-learn,

Algorithm 3 Linear Hinge-Loss Classifier Trained by SGD

Require: $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$, $y_i \in \{-1, +1\}$, learning rate η , regularization λ , epochs T

Ensure: Parameters $\mathbf{w}, b$

```
1: Initialize  $\mathbf{w}, b$ 
2: for  $t = 1, \dots, T$  do
3:   Shuffle the training samples
4:   for each  $(\mathbf{x}_i, y_i)$  do
5:      $s_i \leftarrow \mathbf{w}^T \mathbf{x}_i + b$ 
6:      $m_i \leftarrow y_i s_i$ 
7:     if  $m_i < 1$  then
8:        $\mathbf{w} \leftarrow \mathbf{w} - \eta(\lambda \mathbf{w} - y_i \mathbf{x}_i)$ 
9:        $b \leftarrow b + \eta y_i$ 
10:    else
11:       $\mathbf{w} \leftarrow (1 - \eta \lambda) \mathbf{w}$ 
12:    end if
13:  end for
14: end for
15: return  $\mathbf{w}, b$ 
```

sets the loss to hinge loss and trains a **linear classifier using stochastic gradient descent**. It is therefore closely related to a linear SVM, but the terminology should be precise:

$\text{SVM objective/model} + \text{SGD optimization} \rightarrow \text{SGDClassifier(loss="hinge")}$

The important distinction is that **SVM describes the maximum-margin learning formulation, whereas SGD describes the optimization procedure**. ‘SGDClassifier(loss=”hinge”)’ is not a kernel SVM and does not perform the same optimization procedure as ‘SVC’.

5.11 SGD Training Algorithm for a Linear Hinge-Loss Classifier

Algorithm 4 Stochastic Gradient Descent for a Linear Hinge-Loss Objective

Require: $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$, $y_i \in \{-1, +1\}$, learning rate η , regularization λ , epochs T

Ensure: $\mathbf{w}, b$

```
1: Initialize  $\mathbf{w} \leftarrow \mathbf{0}, b \leftarrow 0$ 
2: for  $t = 1, \dots, T$  do
3:   Shuffle  $\mathcal{D}$ 
4:   for each  $(\mathbf{x}_i, y_i) \in \mathcal{D}$  do
5:      $s_i \leftarrow \mathbf{w}^T \mathbf{x}_i + b$ 
6:      $m_i \leftarrow y_i s_i$ 
7:     if  $m_i < 1$  then
8:        $\mathbf{w} \leftarrow \mathbf{w} - \eta(\lambda \mathbf{w} - y_i \mathbf{x}_i)$ 
9:        $b \leftarrow b + \eta y_i$ 
10:    else
11:       $\mathbf{w} \leftarrow \mathbf{w} - \eta \lambda \mathbf{w}$ 
12:    end if
13:  end for
14: end for
15: return  $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$ 
```

This is a teaching-level form of the algorithm. The exact implementation in a library also involves learning-rate schedules, regularization conventions, stopping criteria, averaging/options, and numerical details.

5.12 SVM vs SGDClassifier(loss=“hinge”) vs LinearSVC

Table 2: Three commonly encountered linear SVM-type choices in scikit-learn.

Property	SGDClassifier(loss=‘hinge’)	LinearSVC	SVC(kernel=‘linear’)
Model	Linear hinge-loss classifier	Linear SVM	Linear SVM
Optimization	Stochastic gradient descent	Dedicated linear SVM solver	General SVM solver
Scalability	Excellent for very large datasets	Good for large datasets	Usually less scalable
Kernel support	No	No	Yes in general SVC; here explicitly linear
Online learning	Yes, via partial fitting	No typical SGD-style online use	No
Training result	Approximate optimization	Dedicated optimization	Dedicated optimization
Main advantage	Speed and scalability	Strong linear-SVM implementation	Flexible SVM framework

Teaching takeaway. Students should not memorize “hinge loss = SGD”. Hinge loss is a **loss function**; SVM is a **maximum-margin learning formulation**; and SGD is an **optimization algorithm**. The same loss can be optimized using different methods, and SGD can also optimize losses other than hinge loss.

5.13 Support Vectors

Only some training samples determine the final boundary. These are the **support vectors**, typically the samples closest to the decision boundary or violating the margin constraints.

CV significance: the classifier is not simply trying to make all training points as far apart as possible; it focuses strongly on difficult boundary samples.

5.14 SVM Training: Conceptual Pseudocode

Actual SVM solvers use optimization algorithms such as SMO or quadratic-programming methods. A teaching-level conceptual algorithm is:

Algorithm 5 Soft-Margin SVM Training

Require: $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, $y_i \in \{-1, +1\}$, $C > 0$

Ensure: Linear decision function $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$

- 1: Solve $\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_{i=1}^N \xi_i$
 - 2: subject to $y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i$ and $\xi_i \geq 0$
 - 3: Identify support vectors from the active margin constraints
 - 4: **return** $\mathbf{w}, b$
-

5.15 SVM Inference

For a linear SVM,

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b.$$

Then

$$\hat{y} = \begin{cases} +1, & f(\mathbf{x}) \geq 0 \\ -1, & f(\mathbf{x}) < 0. \end{cases}$$

Inference is computationally cheap because it requires a dot product and an addition.

6 Kernel SVM: Why and When?

6.1 Why a Kernel Is Needed

A linear boundary may be inadequate if classes are non-linearly arranged. Imagine two classes where one forms a circle around another. No straight line can separate them.

Suppose a nonlinear feature mapping exists:

$$\phi : \mathbb{R}^d \rightarrow \mathcal{H}.$$

In the transformed space $\mathcal{H}$, a linear separator may exist:

$$f(\mathbf{x}) = \mathbf{w}^T \phi(\mathbf{x}) + b.$$

The problem is that explicitly computing $\phi(\mathbf{x})$ can be expensive or even impossible when $\mathcal{H}$ is very high-dimensional.

6.2 Kernel Trick

A kernel computes an inner product in the transformed space directly:

$$k(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j).$$

Thus we can operate as if the data had been mapped to a richer feature space without explicitly constructing that feature vector.

6.3 Dual Form and Kernelized Prediction

The dual SVM representation leads to a decision function of the form

$$f(\mathbf{x}) = \sum_{i \in SV} \alpha_i y_i k(\mathbf{x}_i, \mathbf{x}) + b.$$

Here:

- SV is the set of support vectors;
- $\alpha_i \geq 0$ are learned coefficients;
- $y_i \in \{-1, +1\}$ is the label of support vector i ;
- $k(\mathbf{x}_i, \mathbf{x})$ measures similarity in the implicit feature space.

Only support vectors appear in the final prediction.

6.4 Common Kernels

Linear:

$$k(\mathbf{x}, \mathbf{y}) = \mathbf{x}^T \mathbf{y}.$$

Best when the representation is already approximately linearly separable. Fast and scalable.

Polynomial:

$$k(\mathbf{x}, \mathbf{y}) = (\gamma \mathbf{x}^T \mathbf{y} + r)^p.$$

It models interactions between features. p controls polynomial degree; γ controls scale; r is an offset.

RBF / Gaussian:

$$k(\mathbf{x}, \mathbf{y}) = \exp(-\gamma \|\mathbf{x} - \mathbf{y}\|_2^2).$$

Nearby points have kernel value close to 1; distant points approach 0. The parameter γ controls how local the similarity is.

Sigmoid:

$$k(\mathbf{x}, \mathbf{y}) = \tanh(\gamma \mathbf{x}^T \mathbf{y} + r).$$

Less commonly used in modern CV pipelines.

6.5 When Should a Kernel Be Used?

Use a nonlinear kernel when:

- the feature representation is not linearly separable;
- the dataset is not too large;
- a nonlinear boundary is important;
- handcrafted features have useful nonlinear relationships.

Prefer a linear SVM when the number of samples is large, the feature dimension is high, or the representation is already powerful and approximately linearly separable. Kernel SVMs can become expensive because training and prediction depend on support vectors and kernel evaluations.

7 Multiclass and Multilabel SVM

7.1 Multiclass Classification

Basic SVM is binary. For $C > 2$ classes, common decompositions are:

One-vs-Rest (OvR). Train C binary classifiers. Classifier c separates class c from all other classes:

$$f_c(\mathbf{x}) = \mathbf{w}_c^T \mathbf{x} + b_c.$$

Predict

$$\hat{y} = \arg \max_c f_c(\mathbf{x}).$$

Training cost scales roughly with the number of classes, while each classifier sees all training samples.

One-vs-One (OvO). Train one classifier for every pair of classes:

$$\binom{C}{2} = \frac{C(C-1)}{2}.$$

Each classifier votes for one class and the class with the most votes wins.

7.2 Multilabel Classification

Multiclass means one sample has one class among many. Multilabel means one sample may have several labels simultaneously. For example, an image might contain both *car* and *road*.

A simple SVM-based strategy is **Binary Relevance**:

1. Train one binary SVM for each label.
2. For label ℓ , define target $y_{i\ell} \in \{-1, +1\}$.
3. Obtain a score $f_\ell(\mathbf{x})$ for every label.
4. Predict labels whose scores exceed chosen thresholds.

Formally,

$$\hat{y}_\ell = \mathbf{1}[f_\ell(\mathbf{x}) > t_\ell].$$

This is simple and scalable, but independently trained SVMs do not explicitly model correlations between labels.

8 Decision Trees

8.1 Motivation in Computer Vision

A decision tree recursively divides feature space using simple tests such as

$$x_j < \tau.$$

The tree therefore creates axis-aligned regions. Each leaf stores a class prediction, usually the majority class of training samples reaching that leaf.

This makes decision trees particularly attractive when features have interpretable threshold behaviour, for example:

- color statistics;
- texture measurements;
- shape descriptors;
- object attributes;
- engineered measurements extracted from images.

8.2 Continuous vs Discrete Features

Decision trees can naturally handle both.

Continuous feature. For $x_j \in \mathbb{R}$, test a threshold:

$$x_j \leq \tau.$$

The algorithm searches candidate thresholds and selects one that most improves class purity.

Categorical/discrete feature. For a categorical feature such as texture $\in \{\text{smooth, rough, striped}\}$, the tree can test categories or category subsets, depending on the implementation.

Thus trees do **not** require discretizing continuous CV features beforehand.

8.3 Node Impurity: Gini Index

Suppose a node contains n samples and C classes. Let

$$p_c = \frac{n_c}{n}$$

be the fraction of samples belonging to class c . The Gini impurity is

$$G = 1 - \sum_{c=1}^C p_c^2.$$

If all samples belong to one class, one $p_c = 1$ and $G = 0$: the node is pure.

8.4 Entropy

Another impurity measure is entropy:

$$H = - \sum_{c=1}^C p_c \log_2 p_c.$$

Entropy is zero for a perfectly pure node and larger when classes are mixed.

8.5 Information Gain

Suppose a parent node P is divided into child nodes L and R . Information gain is

$$IG = H(P) - \frac{n_L}{n_P} H(L) - \frac{n_R}{n_P} H(R).$$

The first term measures impurity before the split. The weighted child entropies measure impurity after the split. A good split produces a large reduction.

8.6 Tree Training Algorithm

At each node, consider candidate feature tests and choose the one giving the best impurity reduction.

Algorithm 6 Decision Tree Training for Classification

```
1: function TRAINTREE( $S, depth$ )
2:   if all samples in  $S$  have the same class then
3:     return a leaf with that class
4:   end if
5:   if a stopping condition is satisfied then
6:     return a leaf with  $\arg \max_c \sum_{i \in S} \mathbf{1}[y_i = c]$ 
7:   end if
8:   Generate candidate tests, e.g.  $x_j \leq \tau$  for continuous features
9:   for each candidate split do
10:    Partition  $S$  into  $S_L, S_R$ 
11:     $\Delta I \leftarrow I(S) - \frac{|S_L|}{|S|} I(S_L) - \frac{|S_R|}{|S|} I(S_R)$ 
12:   end for
13:    $(j^*, \tau^*) \leftarrow \arg \max_{j, \tau} \Delta I$ 
14:   Partition  $S$  using  $(j^*, \tau^*)$ 
15:    $T_L \leftarrow \text{TRAINTREE}(S_L, depth + 1)$ 
16:    $T_R \leftarrow \text{TRAINTREE}(S_R, depth + 1)$ 
17:   return node  $(j^*, \tau^*, T_L, T_R)$ 
18: end function
```

Stopping conditions can include maximum depth, minimum samples per node, minimum impurity decrease, or no useful split.

8.7 Geometric Interpretation

If the feature space has two coordinates (x_1, x_2) , a tree might create regions using

$$x_1 < \tau_1, \quad x_2 < \tau_2, \quad x_1 > \tau_2,$$

etc. The resulting decision boundary is piecewise axis-aligned.

This differs from SVM, which produces a hyperplane (or a nonlinear kernel-induced boundary), and KNN, which can create highly irregular local boundaries.

8.8 Advantages, Limitations, and CV Use

Advantages:

- handles continuous and categorical features;
- interpretable decision rules;
- little feature scaling requirement;
- nonlinear decision boundaries are possible through recursive partitioning;
- fast inference for a shallow tree.

Limitations:

- single trees can overfit;
- small data changes can produce different trees;
- axis-aligned splits can be inefficient for diagonal boundaries;
- high-dimensional raw image pixels are usually a poor direct input compared with meaningful engineered features.

9 Bag-of-Visual-Words (BoVW)

9.1 Why BoVW?

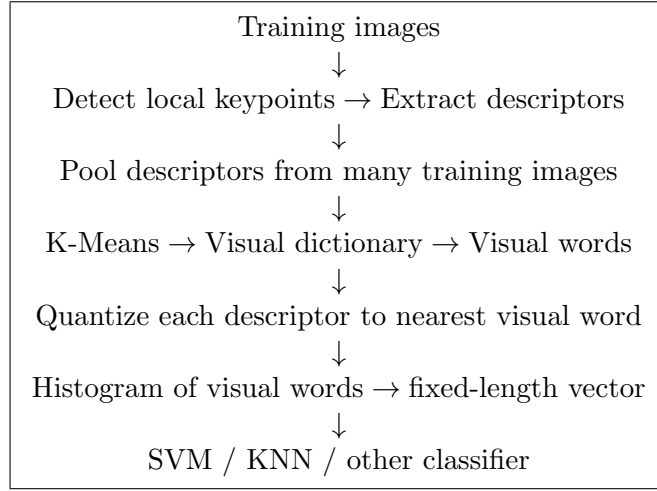
Local feature detectors such as SIFT produce a variable number of descriptors per image. A classifier such as SVM, however, normally expects a fixed-dimensional vector.

BoVW converts

variable number of local descriptors $\longrightarrow$ fixed-length image vector.

It is conceptually analogous to text Bag-of-Words, where a document is represented by word frequencies.

9.2 Complete Pipeline



9.3 Step 1: Extract Local Descriptors

For image I_i , detect M_i local features and compute descriptors

$$D_i = \{\mathbf{d}_{i1}, \dots, \mathbf{d}_{iM_i}\}, \quad \mathbf{d}_{ij} \in \mathbb{R}^d.$$

Different images can have different M_i . This variable length is the problem BoVW solves.

9.4 Step 2: Learn the Visual Dictionary

Pool descriptors from training images:

$$\mathcal{D}_{local} = \{\mathbf{d}_1, \dots, \mathbf{d}_N\}.$$

Run K-Means with K clusters. The learned centroids

$$\mathcal{V} = \{\mathbf{v}_1, \dots, \mathbf{v}_K\}$$

are the visual dictionary.

9.5 Step 3: Vector Quantization

For a descriptor $\mathbf{d}_i$, find its nearest visual word:

$$q(\mathbf{d}_i) = \arg \min_k \|\mathbf{d}_i - \mathbf{v}_k\|_2.$$

Thus a continuous descriptor is replaced by a discrete index $q(\mathbf{d}_i) \in \{1, \dots, K\}$.

9.6 Step 4: Histogram Construction

For image I , let $q(\mathbf{d}_i) = k$ mean that descriptor i is assigned to visual word k . The histogram is

$$h_k = \sum_i \mathbf{1}[q(\mathbf{d}_i) = k], \quad k = 1, \dots, K.$$

Therefore,

$$\mathbf{h} = [h_1, h_2, \dots, h_K]^T \in \mathbb{R}^K.$$

The image now has exactly K numbers regardless of how many local descriptors it originally contained.

9.7 Histogram Normalization

Raw counts depend on the number of detected features. A common normalization is

$$\hat{h}_k = \frac{h_k}{\sum_{j=1}^K h_j + \epsilon}.$$

This converts counts approximately into frequencies.

Other choices include L_1 , L_2 , or TF-IDF-style weighting.

9.8 Step 5: Classification

The final representation is fed into a classifier:

$$\mathbf{h} \rightarrow \text{SVM} \rightarrow \text{predicted object/category}.$$

A common classical pipeline is therefore

$$\text{SIFT} \rightarrow \text{K-Means dictionary} \rightarrow \text{BoVW histogram} \rightarrow \text{SVM}.$$

9.9 BoVW Pseudocode

Algorithm 7 Bag-of-Visual-Words (BoVW) Pipeline

Require: Training images $\{I_n\}_{n=1}^N$, vocabulary size K

Ensure: Visual dictionary $\mathcal{V}$ and trained classifier

- 1: **for** each training image I_n **do**
 - 2: Detect keypoints and compute descriptors $D_n = \{\mathbf{d}_{nj}\}$
 - 3: Add descriptors to pooled set $\mathcal{D}_{local}$
 - 4: **end for**
 - 5: Learn $\mathcal{V} = \{\mathbf{v}_1, \dots, \mathbf{v}_K\}$ using K-Means
 - 6: **for** each training image I_n **do**
 - 7: Initialize $\mathbf{h}_n \leftarrow \mathbf{0} \in \mathbb{R}^K$
 - 8: **for** each descriptor $\mathbf{d}_{nj}$ **do**
 - 9: $q(\mathbf{d}_{nj}) \leftarrow \arg \min_k \|\mathbf{d}_{nj} - \mathbf{v}_k\|_2^2$
 - 10: $h_{n,q(\mathbf{d}_{nj})} \leftarrow h_{n,q(\mathbf{d}_{nj})} + 1$
 - 11: **end for**
 - 12: Normalize $\mathbf{h}_n$ if required
 - 13: **end for**
 - 14: Train classifier on $\{(\mathbf{h}_n, y_n)\}_{n=1}^N$
 - 15: For a test image, repeat extraction and quantization to obtain $\mathbf{h}$
 - 16: **return** $\hat{y}$ from the trained classifier
-

9.10 Major Limitation: Loss of Spatial Information

Two images can have identical visual-word histograms even when their visual words occur at very different locations. BoVW therefore largely discards spatial arrangement.

A common improvement is the **spatial pyramid**: divide the image into regions at multiple resolutions and construct a histogram for each region, then concatenate them.

10 Comparison of Classical Classifiers for Computer Vision

Table 3: Classical ML methods and their roles in CV.

Method	Core idea	Feature-space requirements	Strengths	Weaknesses
KNN	Nearby samples vote	Numerical vectors with meaningful distance; categorical data need suitable distance/encoding	Very simple; no explicit training; flexible boundary	Slow inference; memory-heavy; suffers in high dimensions
SVM	Maximum-margin separation	Continuous numerical vectors; scaling often important	Strong with medium datasets; high-dimensional features; kernels provide nonlinear boundaries	Kernel training can be expensive; hyperparameter tuning required
Decision Tree	Recursive threshold partitioning	Continuous and categorical features; no scaling required	Interpretable; non-linear rules; fast inference	Single trees overfit; unstable; axis-aligned splits
K-Means	Centroid-based clustering	Numerical vectors with meaningful distance	Excellent for clustering/vector quantization/visual dictionaries	Requires K ; sensitive to initialization and geometry

11 Which Method Should Be Used When?

11.1 KNN

Use when the dataset is not huge and a meaningful similarity metric exists. It is particularly useful as a baseline or when local neighbourhood structure is important.

11.2 SVM

Use when there is a good fixed-dimensional feature representation and the dataset is small-to-medium sized. SVMs are historically very strong with handcrafted CV features such as HOG, SIFT/BoVW, texture descriptors, and shape features.

11.3 Decision Tree

Use when interpretability is important, feature thresholds have semantic meaning, or heterogeneous engineered features are available. For stronger predictive performance, ensembles such as Random Forest and Gradient Boosted Trees are often preferred over one tree.

11.4 K-Means

Use when labels are unavailable and clustering is desired, or as a preprocessing/vector-quantization stage. In classical CV, its most recognizable role is learning a visual vocabulary for BoVW.

12 Feature-Space Requirements: A Practical View

Method	Continuous features	Discrete/categorical features	Scaling sensitivity
KNN	Natural	Needs suitable encoding/distance	High
SVM	Natural and usually preferred	Encode categories numerically	Usually high
Decision Tree	Natural	Natural support	Low
K-Means	Required in standard form	Not directly appropriate	High

Table 4: Feature-space requirements and preprocessing considerations.

13 Training Cost vs Inference Cost

Method	Training	Inference	Main bottleneck
KNN	Minimal	Potentially expensive	Distance computation to training set
Linear SVM	Optimization cost	Very low	Training optimization
Kernel SVM	Potentially high	Depends on support-vector count	Kernel matrix/training and SV evaluations
Decision Tree	Moderate	Very low for shallow trees	Tree construction
K-Means	Iterative clustering	Low after dictionary learned	Repeated assignment/update

14 End-to-End Classical CV Example

Consider object-category recognition using local features.

1. Detect SIFT keypoints in training images.
2. Compute a 128-dimensional descriptor for every keypoint.
3. Pool descriptors from the training set.
4. Run K-Means with $K = 500$ to obtain 500 visual words.
5. Quantize each image's descriptors to these 500 words.
6. Construct a 500-dimensional histogram per image.

7. Normalize the histogram.
8. Train a multiclass SVM.
9. For a test image, repeat feature extraction and quantization.
10. Feed the resulting 500-dimensional histogram to the trained SVM.

This example demonstrates the division of labour:

SIFT: What visual information is represented?
K-Means: What recurring local patterns form the vocabulary?
BoVW: How is the variable set of local features summarized?
SVM: How are the resulting image representations separated?

15 Important Practical Lessons for Computer Vision

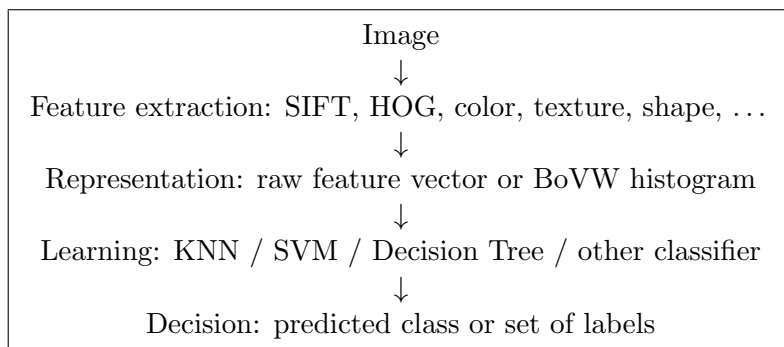
1. **Representation can matter more than classifier choice.** A strong descriptor with a simple classifier can outperform a sophisticated classifier operating on poor features.
2. **Distance is a modelling decision.** In KNN, the distance function defines the meaning of visual similarity.
3. **Scaling matters.** It is especially important for KNN, SVM, K-Means, and other geometry-based methods.
4. **SVM margin is geometric.** The objective is not merely “classify correctly”; it seeks a boundary with a large safety region around it.
5. **Kernels change the representation implicitly.** They allow linear separation in a transformed feature space without explicitly computing that transformation.
6. **Trees partition; SVMs separate.** A tree creates recursive axis-aligned regions, while a linear SVM creates a hyperplane.
7. **BoVW converts variable-length local information into a fixed-length vector.** This is the key reason it can connect local feature methods to conventional classifiers.
8. **Classical pipelines are modular.** Feature extraction, representation, and classification can be replaced independently, making them useful for teaching and controlled experiments.

16 Summary Table

Method	Learning type	Mathematical principle	Typical CV role
K-Means	Unsupervised	Minimize within-cluster squared distance	Visual dictionary / clustering
KNN	Supervised, non-parametric	Neighbourhood voting under a distance metric	Simple recognition baseline / local similarity
SVM	Supervised	Large-margin classification with hinge-loss/constraints	Strong classifier for hand-crafted features
Kernel SVM	Supervised	Linear separation in implicit feature space	Nonlinear classification of fixed features
Decision Tree	Supervised	Recursive impurity-reducing partitions	Interpretable engineered-feature classification
BoVW	Representation pipeline	Quantize local descriptors into visual-word histograms	Fixed-length representation for classical classifiers

17 Final Conceptual Picture

The complete classical CV viewpoint can be summarized as



The central teaching message is therefore:

Classical computer vision separates representation learning from decision making.

The feature extractor defines the coordinates of the feature space;
the machine-learning algorithm defines how that space is partitioned or separated for prediction.